

Windows Virtual Memory

John Wensink

CSC507 Foundations of Operating Systems

Colorado State University – Global Campus

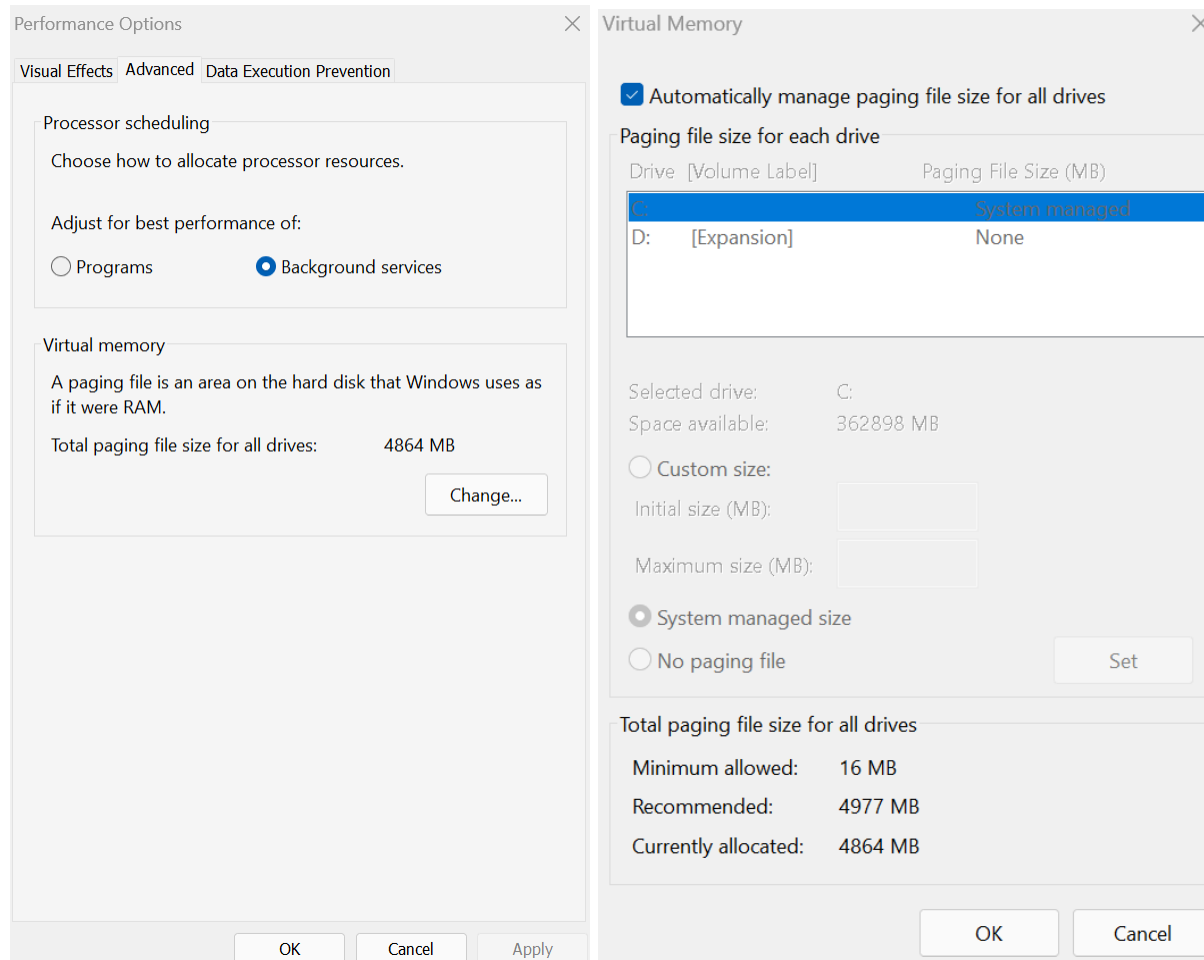
Dr. Joseph Issa

December, 15th, 2024

## Windows Virtual Memory

This week we learned about virtual memory and applied that knowledge by experimenting with its settings in a Windows 11 environment. Virtual memory is a key feature in a modern operating system, allowing for multiprogramming of memory-intensive programs that otherwise would not be possible with a traditional-only approach to memory management. Virtual memory allows a system to use secondary storage (an SSD in this case) to extend the physical RAM capacity. This allows the system to manage workloads that would otherwise exceed the capacity of physical RAM by swapping data pages between main memory and storage (Microsoft, n.d.) Windows relies primarily on paging for virtual memory management (Sinha, 2015), although it can support segmentation to support legacy software, as well as segmentation with paging. By default, my version of Windows was set up to allocate 4864MB to virtual memory, with the page file size automatically managed by the system. From this week's reading, I learned that page size can have a substantial impact on memory performance. A smaller page size reduces internal fragmentation, as it allows memory to be allocated more precisely to the needs of the process. It also enables more processes to have a presence in the page table, increasing the likelihood for a 'ready process' to be available for the processor, which helps to avoid processor downtime. Smaller page sizes will also have the drawback of increased likelihood of page faults, as more pages are required per process, resulting in larger page tables and increased computational overhead for memory management. Conversely, large page sizes are likely to reduce the frequency of page faults, as more data can be stored on a single page. However, this comes with tradeoffs such as increased internal fragmentation, fewer programs

fitting into memory simultaneously, and reduced reference locality benefits, as larger pages may contain unrelated data far from recently accessed locations (Stallings, 2018).

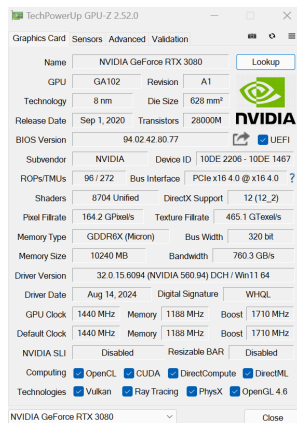
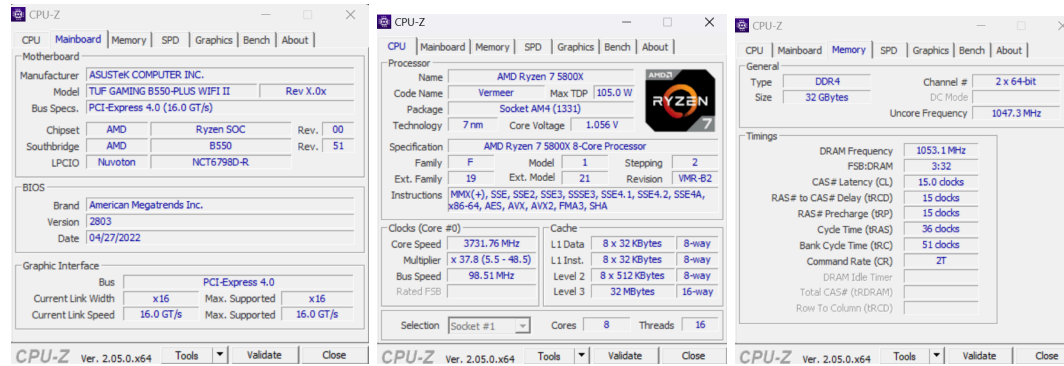


## System Configuration

This week's experimentation will take place on my personal PC, a self-built system running Windows 11. My hardware choices were initially selected for gaming. I don't have much time for gaming these days between school, work, and family, but I have found that the setup has performed excellently for multitasking, virtualization, and programming, which is what I use it for 99% of the time. The main board is an ASUS B550 PLUS WIFI II with an AM4 socket that

holds my AMD Ryzen 7 5800X processor. This class has made me come to appreciate the speed of the 8-core, 16-thread CPU built on a 7nm architecture. I think a lot of people don't realize how absolutely blazingly fast a 3.73GHz clock speed is (this is a mid-range speed, processors can get much faster) and how relatively slow the other I/O components are. For example, the system's main memory, while fast by most standards, runs at a frequency of 1,053 Mhz with a CAS latency of 15 clocks. When compared with the CPU's 3.73 GHz clock speed, it becomes apparent how much faster the processor is at executing instructions versus the memory's ability to deliver data. This difference in speed means that the CPU often ends up waiting for data to arrive from memory, a common bottleneck known as the memory wall. To mitigate this, modern processors like the 5800X incorporate a multi-tiered cache memory (L1, L2, L3), which is much faster than the RAM but has a much smaller size and much higher cost. While the main memory is slower than the CPU, it's still significantly faster than the storage disk. This system has a Samsung 970 EVO Plus 1TB SSD, which runs on a Non-Volatile Memory Express (NVMe) interface. NVMe is a modern protocol designed to take full advantage of the high speeds of modern SSDs by connecting directly to the PCIe bus. This SSD is capable of reads up to 3,500 MB/s and writes up to 3,300 MB/s making it incredibly fast for storage but still significantly slower than the RAM, which operates in the range of tens of GB/s (depending on the configuration) (Cunningham, 2018.) We can begin to see how the relatively slow I/O devices are always struggle to feed the CPU data fast enough to keep it working at full capacity to minimize downtime. When the CPU requests data that isn't found in the cache (cache miss), it must access the system's RAM. If RAM is fully utilized already, the system begins the paging process. With SSDs, we don't have the downside of rotational latency and seeking. However, this improvement

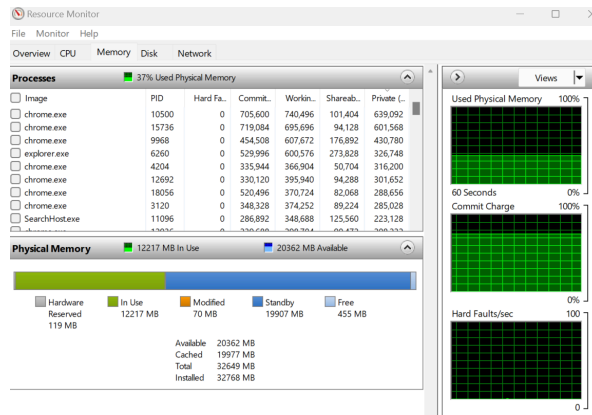
comes at the cost of a limited lifetime of writes, as SSDs have a finite number of program/erase cycles before their memory cells degrade and become unreliable (a fascinating rabbit hole to explore.) In modern SSDs, wear leveling ensures that write/erase cycles get evenly distributed across all memory cells to extend the drive's lifespan as well as overprovisioning (reserving extra space and error-correcting algorithms) helps SSDs maintain performance and reliability over time. This issue is unfortunately exacerbated by using virtual memory, as the frequent paging operations, by their nature, involve frequent, small write and erase cycles between the SSD and RAM. We haven't learned much about video RAM (VRAM) yet, but this system has 10GB of GDDR6X, which provides up to 760 GB/s to allow the GPU to handle large textures, high-resolution gaming, as well as intensive parallel computation independent of the system's main memory.



## Testing

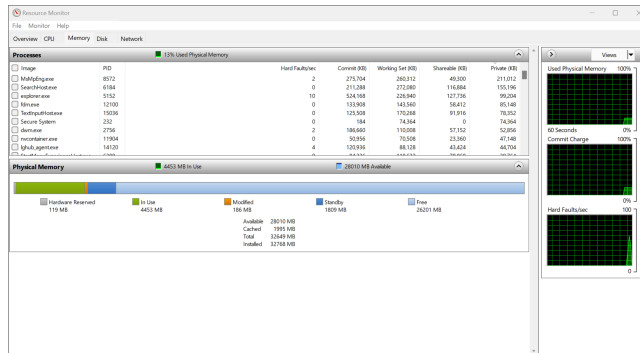
To begin testing, I thought it would be a good idea to create a system restore point before intentionally trying to crash the machine through the VRAM. To that end, I made sure system protection was enabled and created a restore point named “Before Virtual Memory Test.” The default setting was 4864 MB of virtual memory space on the SSD, with the system automatically managing paging file sizes for the C: drive. For this test, we will start by seeing how the system responds to opening 100 Chrome tabs. Since each tab runs as its own process, it seems like it would be a good benchmark to keep consistent across all memory configurations. Task Manager does a sufficient job at showing how the physical RAM is utilized, but to get a better idea of virtual memory usage, we can use Microsoft’s Resource Monitor to look at Hard Faults/s to get

an indirect indication of how virtual memory is performing. Looking at Resource Monitor before the test, we've got about 12.2 GB in use and 19.9 GB in standby with about 0.5 GB free.



I opened 100 Chrome tabs and noticed the in-use memory jump to about 15.1GB with just a few Hard Faults. It doesn't seem to be having as big an effect as I had expected, even when I refreshed every tab at the same time. I suspect Chrome is suspending some of the unused tab processes. I will need to find something a bit more resource-intensive. I think the day has finally come when I get to play video games for homework. I don't have any video editing software on this PC, and gaming is the most resource-intensive process I can think of. I launched Elden Ring, which is likely to be the most resource-intensive game I own. I turned the settings up to max and opened the game in windowed mode, at which point I selected the resolution that took  $\frac{1}{4}$  of the screen (can't resize the window) so that I would be able to keep an eye on Resource Monitor and type/take screenshots. I was disappointed to see that this only took 19.4 GB of memory, and I still have almost 13 GB of standby memory available. At this point, something interesting happened, I went to take a screenshot, and my mouse cursor disappeared. I ended up restarting the system using keyboard commands and plugged in my wired mouse for future testing. I was

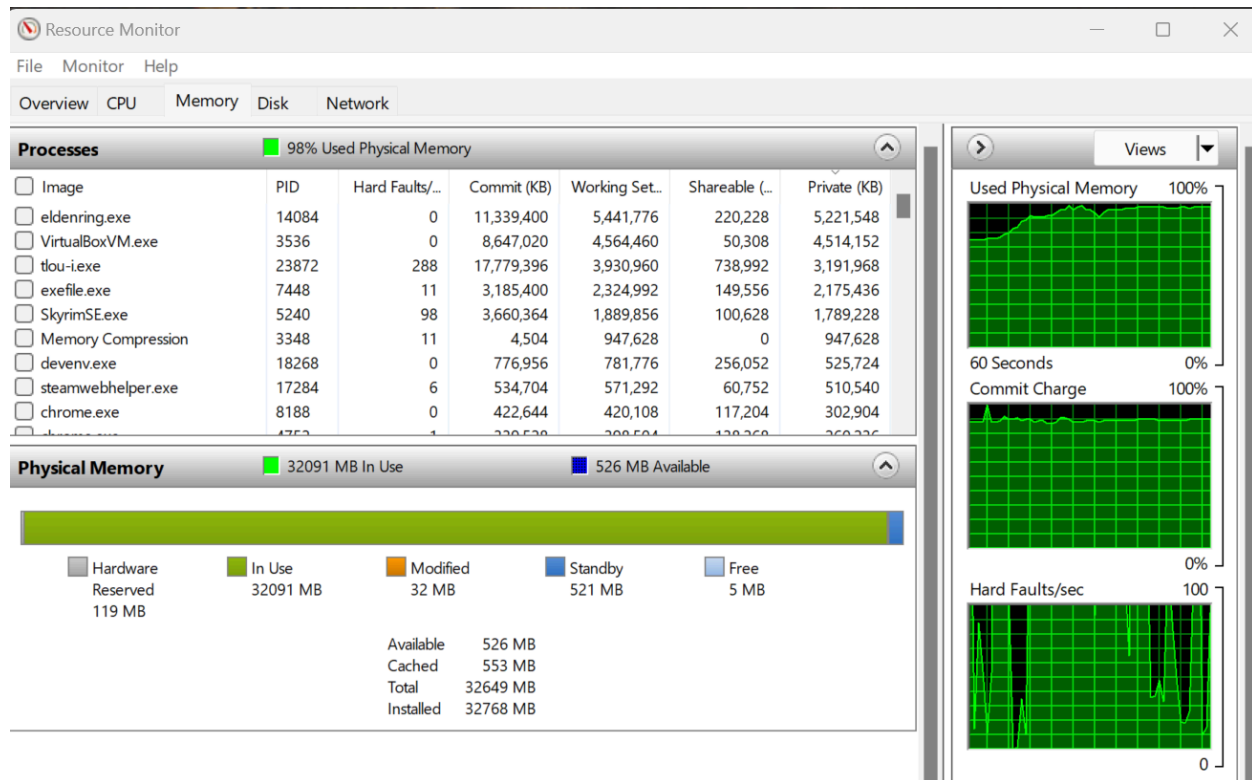
able to get a baseline reading of memory usage, checking Resource Monitor first thing after a restart. Looks like, at a bare minimum, the system is taking up 4.4 GB of RAM with only the startup applications running with 18.0 GB on standby and 26 GB free.



I took the battery out of my wireless mouse and will be running the tests with only the wired mouse plugged into the USB port going forward. I restarted the game in windowed mode at the highest graphics settings, but this time once it gets up and running, I am going to start my Fedora VM, which I have allocated 16 GB of memory, and I expect to see Hard Faults per second start to spike. Fedora seemed to be taking a very long time to load and eventually froze on the boot. With both the VM and the game running together, I was still unable to get the physical memory to max out, so I opened back up the Chrome browser with 100 tabs and refreshed them all simultaneously. Still, I was only able to get the RAM up to 25 GB out of 32 GB. I opened Visual Studio in Windows and started playing 4K videos on YouTube in the VM. I was still unable to get the memory to max out, although the 4K video seemed to get pretty choppy on the VM. I ended up opening every game that I owned that I was able to play in windowed mode and was



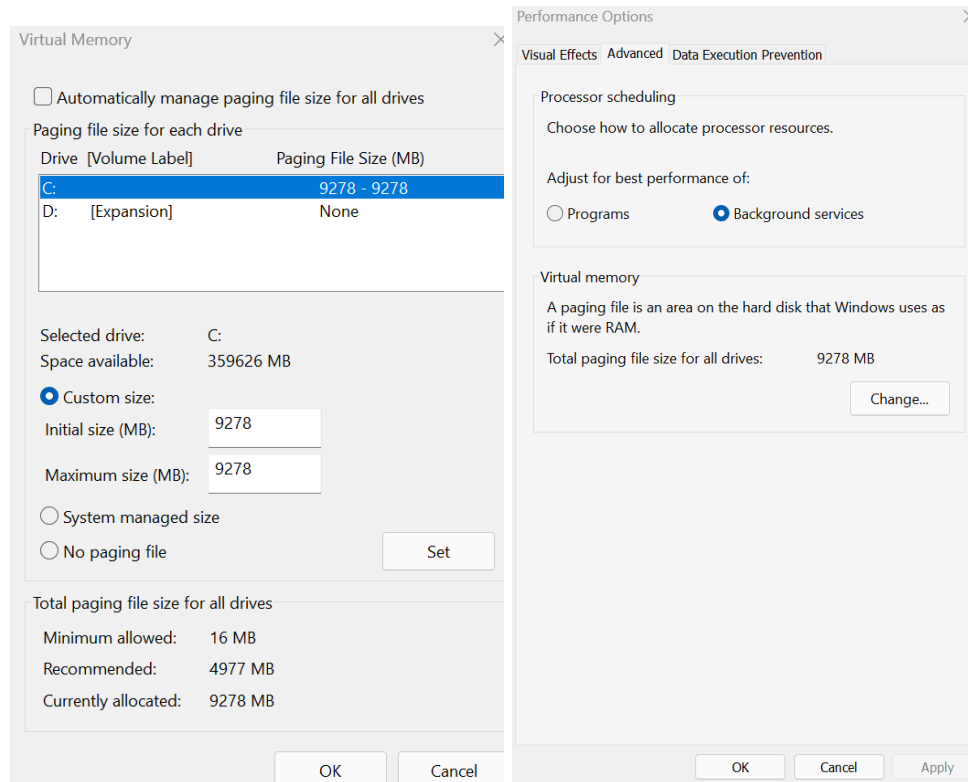
eventually able to get the system to max out RAM and start relying on virtual memory.



I changed the virtual memory settings to allow double the size of the virtual memory and restarted my PC.

## Windows Virtual Memory

10

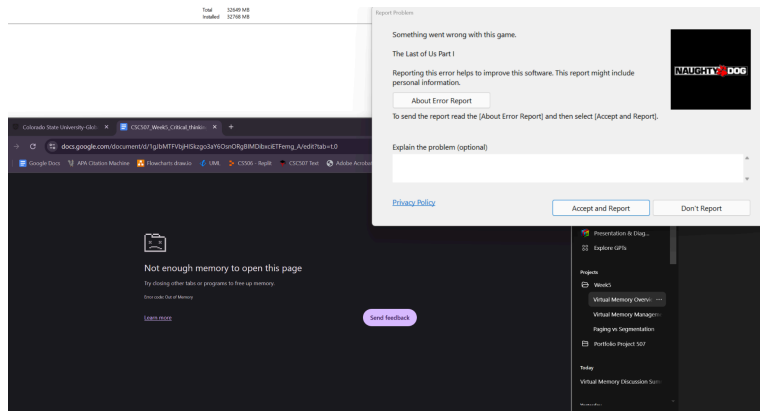


I started to open back up all of the processes to max out the main memory again, not expecting that there would be much difference between the regular system-managed amount vs double the space for virtual memory. I was surprised to notice that I was able to navigate the system much more easily, the choppiness was substantially reduced, and I found I was able to continue typing even while the programs were starting to load. Then, something interesting happened. The game “The Last of Us Part 1” crashed, and my Google Docs sheet where I was typing out my assignment failed to load due to “Not enough memory to open this page.” I suspect when I doubled the amount of virtual memory, my system was no longer dynamically managing virtual memory resources and page size, and the programs weren’t able to get sufficient resources in a timely manner as the system prioritized the memory for other tasks which were no longer

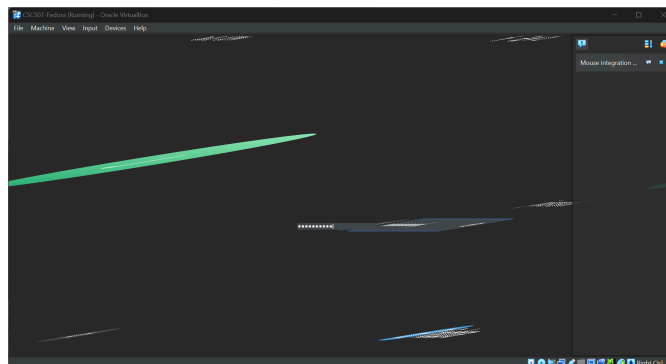
## Windows Virtual Memory

11

ordered in importance. When resources became scarce, the OS had to prioritize the OS and kernel-level functions while sacrificing the user-level processes.



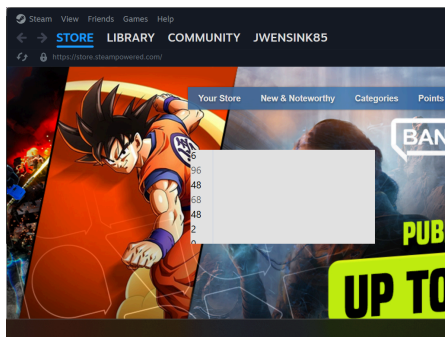
For the next part of the test, I will turn virtual memory off entirely, and I expect that my OS will crash as resources become scarce. Windows warned me that if I choose to disable virtual memory and the system crashes, I may be unable to recover logs to show what may have caused the fatal error. I chose to ignore this warning and proceeded to restart my system to fully disable virtual memory. Upon restart, I noticed that my system was running very choppy even before starting to strain the memory resources and that my VM was experiencing graphical artifacts/display corruption upon startup.



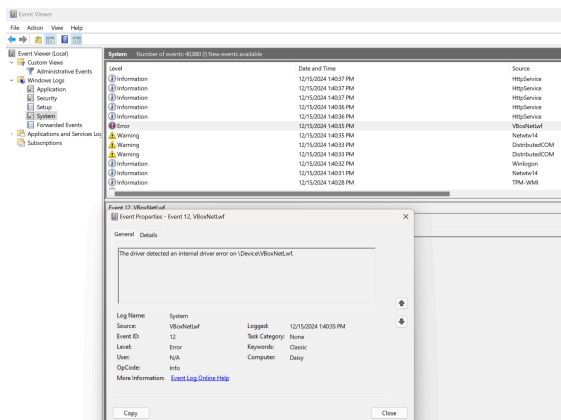
## Windows Virtual Memory

12

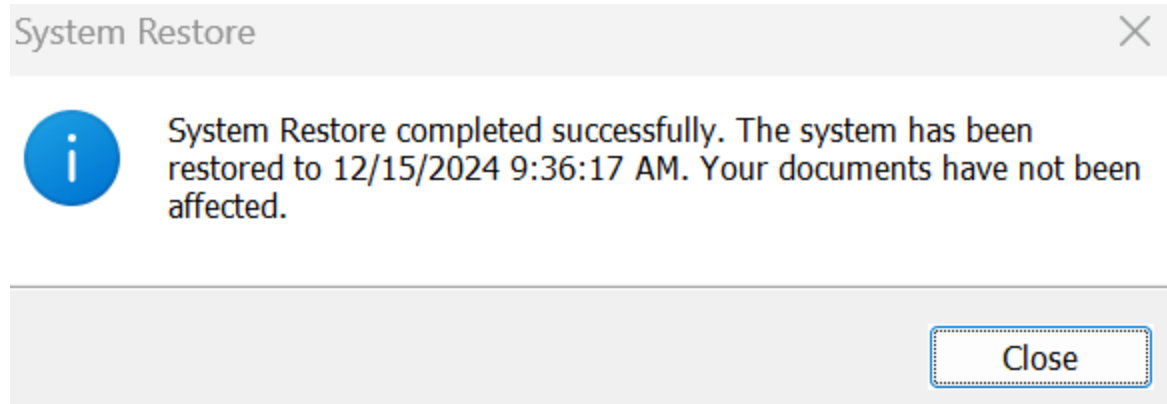
I typed my password and hit enter, and the VM was able to load to the Fedora home screen. I started the HD YouTube video and started to open up games one at a time on my host machine while keeping an eye on Resource Monitor. I was able to launch Elden Ring, EvE Online, and The Last of Us, but things started to get weird pretty quickly. My screen resolution went crazy, and then everything froze. No error messages, but I ended up needing to do a hard reboot to get the PC back up and running.



I opened up Event Viewer, and there was a string of warnings and errors when the system lost control. I believe it was VirtualBox, which was the straw that broke the camel's back under excessive load with virtual memory disabled.



I ended up using the system restore point I created before the testing began, and everything appeared to be functioning normally again.



### Conclusion

This week's experiment with Windows virtual memory showcased its important role in managing system resources under various workloads. Through deliberate testing, I observed how virtual memory supplements physical RAM, enabling the system to handle resource-intensive tasks while maintaining performance. Adjusting virtual memory settings provided insights into the tradeoffs between system responsiveness, resource allocation, error handling, and performance. Notably, doubling the virtual memory space improved navigation and reduced choppiness at first but introduced instability when processes demanded excessive resources. Disabling virtual memory altogether confirmed its critical importance, as the system experienced significant slowdowns and eventually crashed under high load. These findings underscore the delicate balance required in virtual memory management to optimize performance while avoiding resource contention. This exercise deepened my understanding of how modern

operating systems use virtual memory to manage hardware limitations. I am excited to see how this base of knowledge will build into next week's curriculum on process scheduling.

## References

Cunningham, C. (2018, August 7). Data transfer rates compared (RAM vs PCIe vs SATA vs USB vs more!). Logical Increments Blog. Retrieved from <https://blog.logicalincrements.com/2018/08/data-transfer-rates-bandwidth-cpu-ram-pcie-m-2-sata-usb-hdmi/>

Microsoft. (n.d.). Introduction to the page file. Microsoft Learn. Retrieved December 15, 2024, from <https://learn.microsoft.com/en-us/troubleshoot/windows-client/performance/introduction-to-the-page-file>

Sinha, S. (2015, November 11). *Physical and virtual memory in Windows 10*. Microsoft Community. Retrieved December 15, 2024, from <https://answers.microsoft.com/en-us/windows/forum/all/physical-and-virtual-memory-in-windows-10/e36fb5bc-9ac8-49af-951c-e7d39b979938>

Stallings, W. (2018). *Operating systems: Internals and design principles* (9th ed.). Pearson. Retrieved from [https://www.youseficlass.ir/wp-content/uploads/2023/07/William\\_Stallings\\_Operating\\_Systems\\_Internals\\_and\\_Design\\_Principles\\_watermark.pdf](https://www.youseficlass.ir/wp-content/uploads/2023/07/William_Stallings_Operating_Systems_Internals_and_Design_Principles_watermark.pdf)